

MiniVue — Build Vue 3 from Scratch

A framework from the ground up: reactivity, virtual DOM, compiler, components, router, store and SSR

Serik Muradov

Contents

Why Rebuild Vue from Scratch	1
What we'll build	1
The ground rules: no build step	2
How the book and the repository are organized	2
Who this is for	2
The JavaScript You'll Need	3
Values and variables	3
Functions	3
Objects	4
Arrays	4
Destructuring and spread	4
Classes, getters, and setters	5
Set, Map, and WeakMap	6
Symbol	6
Proxy and Reflect	7
Object.is	7
Modules: import and export	7
optional chaining and !!	8
Reactivity	9
The problem in one sentence	9
Two verbs: track and trigger	9
An effect is a function you can re-run	10
Where the dependencies live: targetMap	11
reactive: intercepting an object with Proxy	13
ref: reactivity for a single value	14
computed: lazy and cached	14
Why cleanup is needed: the branching problem	15
watch: observe and get old/new	16
How it all works together	17
Check yourself	17
The Virtual DOM and the Renderer	18
The idea: describe first, compare second	18
VNode: a node as an object	18
The renderer doesn't know about the browser	19
patch: compare old and new	20

Updating in place	20
Comparing lists by key	21
How this ties into reactivity	21
Check yourself	22
Components	23
What a component is	23
The instance: a component's private record	24
setup and the public context	24
Tying into reactivity: render as an effect	24
The scheduler: three changes — one re-render	25
props: data flowing top-down	26
emit: events flowing bottom-up	26
slots: markup from outside	27
provide / inject: through the floors	27
Lifecycle	27
createApp: the entry point	28
Check yourself	28
The Template Compiler	29
Three translation steps	29
Parsing: from text to tree	29
Generation: from tree to code	30
Directives	31
From string to function: with(ctx)	31
How the compiler hooks into the runtime	32
What we simplified	32
Check yourself	32
The Router	33
The idea: URL → component	33
history: where the URL lives	33
The matcher: parsing paths with parameters	34
The reactive current route	34
Navigation and guards	35
RouterView and RouterLink	35
Wiring into the app	36
What we simplified	36
Check yourself	36
The Store	38
The store is reactivity again	38
createPinia: the store container	38
defineStore: two styles	39
One creation path	40
proxyRefs: why store.count, not store.count.value	40
storeToRefs: destructuring without losing reactivity	40
Plugins	41

What we simplified	41
Check yourself	41
SSR and Hydration	42
Rendering to a string	42
createSSRApp	43
The whole server	43
The problem: HTML without life	43
hydrate: adopting the existing DOM	44
What we simplified	44
Check yourself	45
Wrap-up	45
Forms and Bindings	46
class and style bindings	46
v-model: two-way binding	47
Event modifiers	47
What we simplified	48
Check yourself	48
Reactivity, Deeper	49
watchEffect	49
readonly	50
shallowReactive and shallowRef	50
markRaw	50
Wrap-up	51
Directives and Dynamic Components	52
Custom directives	52
How it works	52
Dynamic components	53
v-model on components	54
What we simplified	54
Check yourself	54
Built-in Components	55
Teleport: render somewhere else	55
KeepAlive: hide, don't destroy	56
A slots bug found along the way	57
Async components	57
What we simplified	58
Check yourself	58
Store Deep-Dive and the Capstone App	59
\$patch, \$subscribe, \$reset	59
The final app	60
What we covered	60
What's next	61
Check yourself	61

Why Rebuild Vue from Scratch

There are two ways to understand a tool. The first is to read the manual and start using it. The second is to take it apart and rebuild it yourself. This book is about the second way. We won't learn to write applications with Vue — we'll write Vue itself.

That sounds intimidating, but there's nothing to fear. Inside, Vue isn't magic — it's a handful of simple ideas stacked carefully on top of one another. Reactivity (“the data changed, so the screen updated itself”) is just intercepting reads and writes. Components are functions that return a description of markup. The router is a dictionary of “address → what to show.” Each of these ideas fits in a hundred or two hundred lines that read like plain prose. We'll go through all of them.

What we'll build

By the end of the book we'll have a working mini-framework with the whole ecosystem of real Vue 3:

- **Reactivity** — `ref`, `reactive`, `computed`, `watch`. The core that makes Vue exist in the first place.
- **Virtual DOM and the renderer** — how a description of the interface turns into real elements on the page, and how to update them surgically instead of redrawing everything.
- **Components** — `setup`, `props`, `events` (`emit`), `slots`, `lifecycle hooks`, `createApp`.
- **Template compiler** — how `<div>{{ count }}</div>` turns into a function.
- **Router** — an equivalent of Vue Router: `addresses`, `params`, `RouterView`, `RouterLink`.
- **Store** — an equivalent of Pinia: shared application state.
- **SSR** — server-side rendering and “coming alive” (hydration) on the client.

We deliberately use the same names as real Vue. Our `ref` is called `ref`, our `createApp` is called `createApp`. So everything you understand here maps one-to-one onto the real framework — there's just more code there, because it handles edge cases that we'll sometimes skip for the sake of clarity.

The ground rules: no build step

Normally, to run a Vue project, you install Node, then a bundler (Vite or Webpack), and it “magically” turns your files into something the browser understands. For a beginner that’s the worst possible start: half of what’s happening is hidden inside a tool you don’t yet understand.

So we have a rule: **no build step**. We write in plain ES modules — the standard the browser understands on its own. A code file is wired into the page with a single `<script type="module">` line, and `import` works inside it. Open the page, and the code runs. Everything that happens, happens right in front of you, with no middlemen.

We’ll need Node.js for exactly two things: running tests and spinning up a tiny local server so the browser will allow modules to load (from the `file://` scheme it won’t load them, for security reasons). The framework itself doesn’t depend on Node — until the chapter on SSR, where the server is the whole point.

How the book and the repository are organized

Each layer is, at the same time:

- a **chapter** here in the book, where the idea is explained in words and pictures;
- a **folder in packages/** with working code, heavily commented;
- **tests in test/** that prove the code does what it promises;
- a **demo in playground/** you can open in the browser and poke at by hand.

I recommend keeping the code and the chapter side by side: read the explanation, then open the file and find those exact lines. The code in the book and the code in the repository are literally the same lines.

Who this is for

For anyone who wants to understand, not just use. If you’ve never written JavaScript, don’t worry: the next chapter gives you exactly the minimum of the language you need to read our code. If you already write Vue but it’s a “black box” to you, this book opens the box.

One warning: don’t try to memorize everything at once. Reactivity in the first chapter feels like a puzzle at first — that’s normal. Get to the demo, poke the buttons, come back to the code. The idea will click into place. And once it does, everything else in Vue becomes an obvious consequence.

Here we go.

The JavaScript You'll Need

This chapter isn't a full language course — it's a glossary. It covers exactly the constructs that show up in our code, and nothing extra. If you already know JavaScript, skim it and move on to reactivity. If you don't, read it once and then come back to it as a reference whenever an unfamiliar symbol turns up in the code.

Values and variables

A value is a piece of data: the number 42, the string "hello", the boolean true or false, or “nothing” — null and undefined. A variable is a name for a value.

```
const name = 'Anya' // const – a name you can't reassign
let count = 0       // let – a name you can change
count = count + 1   // now count is 1
```

const doesn't mean “the value is immutable” — it means “you can't reassign this name.” You can still change the object a const name points to; you just can't say name = another object. Default to const, and reach for let only when you genuinely need to reassign.

Functions

A function is a reusable block of actions that you can run many times, feeding it different inputs (arguments).

```
function double(x) {
  return x * 2
}
double(21) // 42
```

There's a shorter form — the arrow function. We'll see it constantly.

```
const double = (x) => x * 2      // single expression – return is implied
const log = (msg) => {           // body in braces – you write return yourself
  console.log(msg)
}
const now = () => Date.now()     // no arguments – empty parentheses
```

You can pass a function to another function as an argument — that’s a “callback,” a handler function. All of reactivity is built on taking someone else’s function and deciding when to call it:

```
effect(() => {
  console.log('rerun me when the data changes')
})
```

Objects

An object is a set of “key: value” pairs. Keys (properties) are read and written with a dot.

```
const user = { name: 'Anya', age: 30 }
user.name    // 'Anya' – reading
user.age = 31 // writing
user.city = 'Almaty' // adding a new property
```

Remember the phrases **property read** and **property write** — the whole first chapter revolves around them. Reactivity is intercepting exactly these two operations.

Arrays

An array is an ordered list of values. Indexing starts at zero.

```
const items = [10, 20, 30]
items[0]     // 10
items.length // 3
items.push(40) // add to the end → [10, 20, 30, 40]
items.reduce((sum, x) => sum + x, 0) // 100 – "fold" into a single number
```

reduce walks the elements and accumulates a result: start at 0, add each element’s value. That’s how we’ll compute a cart total.

Destructuring and spread

You can “unpack” an object or array into separate variables:


```
const { name, age } = user      // name = 'Anya', age = 31
const [first, second] = items  // first = 10, second = 20
```

Three dots ... mean “spread the rest.” In our code you’ll see it as “make a copy”:

```
const copy = [...items]        // a new array with the same elements
```

This line will save us in reactivity: before iterating over a set of effects, we copy it so that changes made during iteration don’t break the loop.

Classes, getters, and setters

A class is a template for creating objects with shared behavior. `new` creates an instance, `constructor` sets up the initial state, and `this` inside means “this particular instance.”

```
class Counter {
  constructor(start) {
    this.value = start  // instance field
  }
  increment() {
    this.value++
  }
}
const c = new Counter(5)
c.increment()
c.value // 6
```

A special pair is the **getter** and **setter**. These are methods that look like an ordinary property from the outside, but run code on read/write. That’s exactly how `ref` works: reaching for `.value` looks like a field, but actually runs a function.

```
class Box {
  constructor(v) { this._v = v }
  get value() {      // runs on a READ of box.value
    console.log('read')
    return this._v
  }
  set value(newV) {   // runs on a WRITE of box.value = ...
    console.log('wrote')
    this._v = newV
  }
}
```

```
}  
const box = new Box(1)  
box.value    // prints "read", returns 1  
box.value = 2 // prints "wrote"
```

The underscore in `_v` is just a convention meaning “this is an internal field, don’t touch it from outside.” The language doesn’t enforce it, but that’s the custom.

Set, Map, and WeakMap

Besides arrays and objects, we need three special collections.

Set — a collection of unique values. Add the same thing twice, and it’s stored once. Perfect for “the list of effects subscribed to a property”: one effect shouldn’t land there twice.

```
const s = new Set()  
s.add('a'); s.add('a'); s.add('b')  
s.size    // 2  
s.has('a') // true  
s.delete('b')
```

Map — like an object, but the key can be anything, including another object. That’s critical for us: we’ll be linking a reactive object to its dependencies.

```
const m = new Map()  
m.set('key', 123)  
m.get('key') // 123
```

WeakMap — the same as Map, but with “weak” object keys: if nothing else references a key object anymore, the garbage collector is free to remove it along with its entry. This way the dependency store doesn’t keep objects alive that are no longer needed.

Symbol

`Symbol('name')` creates a unique marker value that’s guaranteed not to collide with any string key. We use symbols as internal “hidden” properties — for example, a marker meaning “this object is already reactive,” which can’t be confused with the user’s real data.

```
const IS_REACTIVE = Symbol('isReactive')  
obj[IS_REACTIVE] // no ordinary property will ever land here by accident
```

Proxy and Reflect

Here's the main tool of reactivity. **Proxy** wraps an object and lets you intercept operations on it — first and foremost reads (get) and writes (set).

```
const raw = { count: 0 }
const proxy = new Proxy(raw, {
  get(target, key) {
    console.log('reading', key)
    return target[key]
  },
  set(target, key, value) {
    console.log('writing', key, '=', value)
    target[key] = value
    return true // set must return true on success
  },
})
proxy.count // prints "reading count", returns 0
proxy.count = 5 // prints "writing count = 5"
```

reactive grows out of this tiny bit of interception: in get we'll call “remember who read this,” and in set we'll call “wake up whoever read it.”

Reflect — a set of functions that mirror the standard operations. `Reflect.get(target, key, receiver)` is the same as `target[key]`, but it correctly passes this to getters and works with inheritance. Inside a Proxy the right thing is to use `Reflect`, not `target[key]` directly — otherwise tricky objects will break.

Object.is

Comparing “did the value change?” A plain `===` works almost everywhere, but it has some historical quirks: `NaN === NaN` gives false, even though it's “the same thing.” `Object.is` fixes those. We'll ask “is the value really different?” via `!Object.is(old, new)` so we don't wake up the interface when assigning the same value.

Modules: import and export

Code is split into module files. A file shares things by export-ing them and grabs other files' things via import.

```
// effect.js
export function track() { /* ... */ }
export let activeEffect = undefined

// reactive.js
import { track, activeEffect } from './effect.js'
```

The path is written with the `.js` extension and with `./` for neighboring files — that’s what the browser ES modules standard requires, and we use it without a bundler.

optional chaining and !!

`a?.b` reads `a.b`, but if `a` is `null` or `undefined`, it calmly returns `undefined` instead of throwing. `!!x` turns any value into an honest `true/false` (two “nots”: the first inverts, the second flips it back). When you see `return !(value && value.__isRef)`, it just means “return true if value has the `__isRef` flag.”

This glossary is enough to read any line of our framework. Don’t memorize it all — keep this chapter handy and come back as needed. Now, on to the heart of Vue.

Reactivity

Everything in Vue rests on one ability: data changes, and whatever depends on it updates by itself. You change `count`, and the number on screen becomes different, even though you never wrote “find the element and update its text.” This chapter is about how that works. We’ll build reactivity in full, and from here on the whole framework is just a layer on top of it.

The code for this chapter lives in `packages/reactivity/`. Tests are in `test/reactivity.test.mjs`, and the live demo is `playground/01-reactivity.html`.

The problem in one sentence

We want to be able to write this:

```
const count = ref(0)

effect(() => {
  document.title = 'Clicks: ' + count.value
})

count.value = 5  // and the tab title becomes "Clicks: 5" on its own
```

The idea is simple. There’s data (`count`) and there’s a function that uses it (`effect`). We want the function to re-run when the data changes — by itself, without our involvement. So the system needs to somehow learn that this function “depends” on `count`. And it can learn that at exactly one moment — when the function reads `count.value`. Everything is built around this idea.

Two verbs: track and trigger

Remember these two words; the rest of the book revolves around them.

- **track** — happens on a **read**. The meaning: “some function is running right now, and it read this value — let’s remember the connection.”

- **trigger** — happens on a **write**. The meaning: “the value changed — let’s find every function that read it and re-run them.”

Read — remember the dependency. Write — use the dependency. Everything else is details of how those dependencies are stored.

An effect is a function you can re-run

The function that has to re-run on changes is what we call an **effect**. So the system can manage it, we wrap it in an object. That way we can attach bookkeeping data to the function.

The key variable is `activeEffect`. While an effect’s body is running, this variable holds that effect. Any data read at that moment knows “whom” to assign the dependency to. Outside effects it’s undefined, and a read tracks nothing.

```
// packages/reactivity/effect.js
export let activeEffect = undefined
const effectStack = []
```

Why a stack? Effects can be nested (a component, and inside it a computed value). When the inner effect finishes, the outer one has to become “active” again. The stack handles exactly that: push on entry, pop on exit.

The effect itself is the `ReactiveEffect` class:

```
export class ReactiveEffect {
  constructor(fn, scheduler = null) {
    this.fn = fn
    this.scheduler = scheduler
    this.deps = []
    this.active = true
  }

  run() {
    if (!this.active) return this.fn()
    cleanup(this) // forget old dependencies
    try {
      effectStack.push(this)
      activeEffect = this // "I'm listening now"
      return this.fn() // run it - tracks happen inside
    } finally {
      effectStack.pop()
    }
  }
}
```

```
    activeEffect = effectStack[effectStack.length - 1]
  }
}
```

The magic is in the `run` method. Before calling the function it declares itself active (`activeEffect = this`). While the function runs and reads data, every read sees this `activeEffect` and records the dependency. As soon as the function finishes, we pop ourselves off the stack and restore whoever was active before us. The `try/finally` wrapper guarantees the active effect is restored even if an error happens inside.

We'll cover scheduler and cleanup shortly — first let's see how the dependencies are stored.

Where the dependencies live: targetMap

We need to store which effects depend on which property of which object. The structure is three levels deep:

```
targetMap: WeakMap {
  reactiveObject → depsMap: Map {
    'property' → dep: Set(effect1, effect2, ...)
  }
}
```

Read it as: “for this object, for this property of it — here is the set of effects that read it.” The top level is a `WeakMap`, so forgotten objects aren't held in memory. The middle level is a `Map` keyed by property name. The bottom level is a `Set` of effects (unique, no duplicates).

The `track` function fills this structure in:

```
export function track(target, key) {
  if (!activeEffect) return // no one is listening – bail out

  let depsMap = targetMap.get(target)
  if (!depsMap) targetMap.set(target, (depsMap = new Map()))

  let dep = depsMap.get(key)
  if (!dep) depsMap.set(key, (dep = new Set()))

  trackEffects(dep) // link the active effect to this dep
}
```

```
export function trackEffects(dep) {  
  if (!activeEffect) return  
  dep.add(activeEffect)           // dep knows about the effect  
  activeEffect.deps.push(dep)     // the effect knows about the dep  
}
```

Notice the two-way link in `trackEffects`. The `dep` remembers the effect — so it can wake it up later. But the effect also remembers the `dep` (in its `deps` array) — which we'll need for cleanup, covered below.

The reverse operation is `trigger`:

```
export function trigger(target, key) {  
  const depsMap = targetMap.get(target)  
  if (!depsMap) return           // no one read this object  
  const dep = depsMap.get(key)  
  if (!dep) return  
  triggerEffects(dep)  
}  
  
export function triggerEffects(dep) {  
  const effects = [...dep]       // a copy! (explained below)  
  for (const effect of effects) {  
    if (effect === activeEffect) continue // don't wake ourselves  
    if (effect.scheduler) effect.scheduler()  
    else effect.run()  
  }  
}
```

Two subtleties, each of which would otherwise cost hours of debugging:

1. **A copy of the set** (`[...dep]`) before iterating. When an effect re-runs, its `cleanup` removes it from `dep` and `track` immediately adds it back. Mutating a `Set` you're currently looping over is a sure way to get an infinite loop or a skipped element. We iterate over the copy — let the original change freely.
2. **`if (effect === activeEffect) continue`** — protection against self-re-running. If you write `count.value++` inside an effect, it reads and writes `count` at the same time. Without this line it would wake itself up forever.

reactive: intercepting an object with Proxy

Now let's make a plain object call `track` and `trigger` automatically. The tool is `Proxy`. In the `get` trap we call `track`, in the `set` trap we call `trigger`.

```
// packages/reactivity/reactive.js
export function reactive(target) {
  if (!isObject(target)) return target
  // ... guard against double-wrapping ...

  return new Proxy(target, {
    get(obj, key, receiver) {
      const result = Reflect.get(obj, key, receiver)
      track(obj, key) // "I was read"
      if (isObject(result)) return reactive(result) // depth on demand
      return result
    },
    set(obj, key, value, receiver) {
      const oldValue = obj[key]
      const result = Reflect.set(obj, key, value, receiver)
      if (hasChanged(oldValue, value)) trigger(obj, key) // "I was changed"
      return result
    },
  })
}
```

Three decisions worth spelling out.

Deep reactivity on demand. If a read property turns out to be an object itself, we wrap it in `reactive` right at that moment — not ahead of time, recursively across the whole tree (that's expensive and sometimes harmful), but lazily, when something actually reaches for it. So `state.user.name` is reactive at every level, but you don't pay the price until `user` is touched.

The `hasChanged` check. We call `trigger` only if the value is actually different. Assigning the same value shouldn't wake the interface.

Reflect instead of `obj[key]`. On objects with getters and inheritance, direct access loses the correct this. `Reflect.get/set` with `receiver` preserves it. On plain objects there's no difference, but it's worth getting used to the correct form.

ref: reactivity for a single value

Proxy intercepts an object's properties. But how do you make a plain number reactive? A number has no properties to intercept. The solution: put the value inside an object, in a `.value` field, and watch reads and writes of that field through a getter and a setter.

```
// packages/reactivity/ref.js
class RefImpl {
  constructor(value) {
    this._value = convert(value) // an object inside a ref is made reactive too
    this._rawValue = value
    this.dep = new Set() // its own set of effects (a ref has one "property")
    this.__isRef = true
  }
  get value() {
    if (activeEffect) trackEffects(this.dep) // read → track
    return this._value
  }
  set value(newValue) {
    if (hasChanged(toRaw(newValue), this._rawValue)) {
      this._rawValue = toRaw(newValue)
      this._value = convert(newValue)
      triggerEffects(this.dep) // write → trigger
    }
  }
}
```

This is why you always write `.value` on a ref: it's not a whim, it's the only hook through which the getter and setter can step in and call `track`/`trigger`. A ref keeps its own `dep` right inside the object — it doesn't need `targetMap`, since it has exactly one “property.”

There's a handy helper, `proxyRefs` — it unwraps `.value` automatically so that in templates you can write `count` instead of `count.value`. Later the component layer applies it to the result of `setup`, and `.value` in markup disappears. Its code is short; take a look in `ref.js`.

computed: lazy and cached

A computed value is a formula on top of other reactive data, with two useful properties: it's evaluated **lazily** (only when the result is asked for) and it's **cached** (until the dependencies change, a repeat request returns the ready value).

It's assembled from parts we already have: an effect with lazy startup and a scheduler, plus a “dirty” flag.

```
// packages/reactivity/computed.js
class ComputedRefImpl {
  constructor(getter) {
    this._dirty = true           // whether a recompute is needed
    this.dep = new Set()
    this.__isRef = true
    this.effect = new ReactiveEffect(getter, () => {
      if (!this._dirty) {       // a dependency changed →
        this._dirty = true     // mark as dirty
        triggerEffects(this.dep) // and wake whoever read the computed
      }
    })
  }
  get value() {
    if (activeEffect) trackEffects(this.dep)
    if (this._dirty) {         // recompute only if dirty
      this._value = this.effect.run()
      this._dirty = false
    }
    return this._value
  }
}
```

Here the **scheduler** (the second argument to `ReactiveEffect`) comes into play for the first time. An ordinary effect re-runs immediately when a dependency changes. But computed shouldn't evaluate immediately — it should evaluate lazily. So instead of recomputing, its scheduler only raises the `_dirty` flag and wakes the readers. The actual recompute happens in `get value`, and only if someone actually asks for the result. Read it twice without changes, and the second read comes from the cache — the getter doesn't run.

Why cleanup is needed: the branching problem

Back to cleanup, which run calls before every run. Consider an effect with a condition:

```
effect(() => {
  out.value = show.value ? a.value : b.value
})
```

When `show` is true, the effect reads `show` and `a`, but **not** `b`. So it shouldn't subscribe to `b` — otherwise a change to `b`, which currently affects nothing, would re-run it for no reason. But on the next run `show` might become false, and then it needs `b`, not `a`.

Conclusion: before every run the effect must forget all its previous dependencies and collect them again — only the ones actually read this time. That's what `cleanup` does:

```
function cleanup(effect) {  
  for (const dep of effect.deps) dep.delete(effect)  
  effect.deps.length = 0  
}
```

It walks every `dep` where the effect is recorded (this is why the effect kept back references in `deps`), removes itself from them, and clears its own list. After `cleanup` the effect is “subscribed to nothing” — and then `fn()` subscribes it again, but this time based on what's actually read. In the tests this is covered by the “branching — unsubscribe from an unused dependency” case: after switching branches, a change to the old variable no longer wakes the effect.

watch: observe and get old/new

effect just re-runs a function. `watch` is a layer on top that, on a change, calls a callback and passes it the new and old values. The source can be a `ref`, a getter function, or an entire reactive object.

The idea: reduce any source to a getter function, wrap it in an effect, and put the whole “reaction” in the scheduler — it computes the new value and calls the callback.

```
// packages/reactivity/watch.js (abridged)  
let oldValue  
const job = () => {  
  const newValue = effect.run()  
  callback(newValue, oldValue)  
  oldValue = newValue  
}  
const effect = new ReactiveEffect(getter, job)  
oldValue = effect.run()           // first run: collect dependencies, remember the start
```

For a reactive object, to watch it “deeply,” a traverse walk is used — it recursively reads every nested property, thereby subscribing the effect to each level. `immediate: true` makes the callback fire right away, without waiting for the first change.

How it all works together

Let's put the picture together with an example from the demo. There's `count = ref(0)`, there's a computed `doubled = count.value * 2`, and there's an effect that writes both numbers into the DOM.

1. The effect runs for the first time. `activeEffect` is it. It reads `count.value` ($\rightarrow$ track: `count.dep` remembered the effect) and `doubled.value`.
2. Reading `doubled.value` while `_dirty = true` runs its inner effect, which reads `count.value` ($\rightarrow$ `count.dep` also remembered the computed effect). The result is cached, `_dirty = false`.
3. The user clicks "+1": `count.value = 1`. The ref setter calls `triggerEffects`.
4. `count.dep` now has two "listeners": our DOM effect and the computed effect. The computed has a scheduler — it sets `_dirty = true` and wakes the readers of `doubled`. The DOM effect has no scheduler — it just re-runs.
5. The DOM effect reads `doubled.value`. It's `_dirty`, so it recomputes (2) and caches. The numbers on screen are updated.

We didn't write a single line of "find the element, set the text" on the click. We only described the dependencies, and the system pulled the change through the chain by itself. That's the whole essence of Vue — and we've just built it.

Check yourself

Run the tests:

```
npm test
```

Thirteen checks in `test/reactivity.test.mjs` confirm every property: `ref` and `reactive` react, `computed` is lazy and caches, `cleanup` unsubscribes from the unneeded, `watch` hands back old and new. Then open the demo:

```
npm run serve
```

```
# http://localhost:5173/playground/01-reactivity.html
```

Click the buttons, watch the console to see the effect fire. Once reactivity has "clicked" for you, move on to the next layer — the virtual DOM. There we'll learn to turn data not into `document.title`, but into whole trees of elements, and to update them surgically.

The Virtual DOM and the Renderer

In the last chapter, reactivity updated `document.title` — a single line. A real interface is a tree of hundreds of elements, and rebuilding it whole on every change is too expensive: the browser stutters, input fields lose focus, scroll position jumps. We need a way to touch only what actually changed. That’s the renderer’s job, and the **virtual DOM** is what makes it possible.

The code for this chapter lives in `packages/runtime-core/` (the platform-independent core) and `packages/runtime-dom/` (the browser binding). Tests are in `test/renderer.test.mjs`, and the demo is `playground/02-vdom.html`.

The idea: describe first, compare second

Touching the page directly (`document.createElement`, `el.appendChild`) is slow and verbose. So we take a smarter route. First we describe what the interface should look like, using plain objects. Such a description object is called a **VNode** (virtual node). Then we hand that description to the renderer, and it figures out which real operations on the page are needed.

The payoff: when the data changes, we build a new description and place it next to the old one. The renderer compares the two descriptions (this is called a **diff**) and applies only the difference to the real page. No “tear it all down and repaint.”

VNode: a node as an object

A single VNode looks like this:

```
{
  type,      // 'div' (tag), or Text, Fragment, or a component
  props,     // { id, class, onClick, ... }
  children,  // a string, an array of child VNodes, or null
  key,       // key for matching items in lists (more on this below)
```

```
el,      // reference to the real node – the renderer sets it on mount
}
```

Text and Fragment are special marker types (Symbol) with no tag. Text is just text. Fragment is a group of nodes without a shared wrapper (for when you need to return several elements without wrapping them in an extra `<div>`).

Writing these objects by hand is awkward, so there’s a function `h` (from “hyperscript”):

```
h('div', { id: 'app' }, 'hello')
h('ul', [ h('li', 'one'), h('li', 'two') ])
```

`h` can tell whether you passed props or went straight to children, and either way it assembles a correct `VNode`. `h` is exactly what our template compiler will call in layer 4 — so `<div>{{ x }}</div>` ends up becoming a call to `h`.

The renderer doesn’t know about the browser

The key architectural decision is that the renderer isn’t tied to the DOM. It receives every operation on real nodes from the outside, through an options object (we call it `nodeOps`): “create an element,” “insert,” “remove,” “change text.” The diff algorithm itself is independent of them.

```
export function createRenderer(options) {
  const {
    createElement: hostCreateElement,
    insert: hostInsert,
    remove: hostRemove,
    patchProp: hostPatchProp,
    // ...
  } = options
  // ... all of the diff lives inside ...
  return { render }
}
```

Why this abstraction? Because the same renderer then works anywhere. For the browser, the operations come from `runtime-dom` (files `nodeOps.js` and `patchProp.js`). For tests, an invented in-memory tree (`test/helpers/testHost.mjs`), where we exercise the diff without any browser. For the server in layer 7, there will be its own implementation. One algorithm, three environments. Real Vue is built exactly this way.

patch: compare old and new

The central function is `patch(n1, n2, container, anchor)`. It compares the old node `n1` and the new node `n2`:

- `n1 === null` — the node appeared for the first time, so we **mount** it;
- `n1.type !== n2.type` — the nodes are incompatible (you can't turn a `div` into a `span`), so we remove the old one and mount the new one from scratch;
- otherwise — the types match, so we **update in place**: the most common and cheapest path.

`anchor` is the node to insert before. It's what lets us land in the right spot among siblings; `null` means "at the end."

From there `patch` branches by the new node's type: `text`, `fragment`, `element`, or `component`. For an element appearing for the first time, `mountElement` runs:

```
function mountElement(vnode, container, anchor) {
  const { type, props, children } = vnode
  const el = (vnode.el = hostCreateElement(type)) // create and remember the node
  for (const key in props) {
    hostPatchProp(el, key, null, props[key])      // set attributes
  }
  if (typeof children === 'string') {
    hostSetElementText(el, children)              // text content
  } else if (Array.isArray(children)) {
    mountChildren(children, el, null)             // or children recursively
  }
  hostInsert(el, container, anchor)               // insert into the parent
}
```

Notice `vnode.el = ...`. We store the reference to the real node right on the `VNode`. On the next update, the new `VNode` inherits this `el` — that's how we reuse the existing element instead of creating a new one.

Updating in place

When the types match, `patchElement` is called: it takes `el` from the old `VNode`, compares attributes (`patchProps`) and children (`patchChildren`).

`patchProps` walks the new props (updating changed ones, adding new ones) and the old props (removing ones that disappeared). The detail that's easy to forget is removal: if a node had a `class` and the new description doesn't, the attribute has to come off.

`patchChildren` handles the “was → became” cases. Children can be text, an array, or empty. Most combinations are simple (became text — set the text; was text, became an array — clear it and mount). One case is hard and deserves its own discussion — an array turning into an array.

Comparing lists by key

Picture a list of a thousand rows, and one gets added at the front. The naive approach would compare pairwise: first with first, second with second — and conclude that all thousand changed. A thousand needless edits instead of one insertion.

The rescue is **keys**. We ask you to tag list items with a unique key (usually an id from the data). Then the renderer matches nodes by key rather than by position: “the node with key 42 was third, now it’s first — don’t recreate it, just move it.”

The `patchKeyedChildren` algorithm (the same one Vue 3 uses) works in several passes:

1. **From the start:** while keys match, update nodes in place and move forward.
2. **From the end:** the same from the tail of the list. These two passes instantly dispatch the most common cases — items added or removed at the start or the end.
3. If only new nodes remain after that — mount them.
4. If only old nodes remain — unmount them.
5. If the middle is shuffled — build a “key → new index” map, update the matched nodes, remove the extras, and move the rest.

The fifth step hides one more optimization. Not every node needs moving — some are already in the correct relative order. To find the largest such group, we compute the **longest increasing subsequence** (LIS, the `getSequence` function). Nodes in it we leave untouched, moving only the others. That keeps the number of DOM rearrangements minimal.

You can verify this in the test “reversing a list keeps the nodes”: we record the real node objects before the rearrangement and confirm that after `[a,b,c] → [c,b,a]` they’re the same objects, just reordered. Not one is recreated — which means focus, selection, and any other live state would have survived.

How this ties into reactivity

The renderer on its own knows nothing about reactivity — it’s simply told “draw this.” One effect connects the two layers. Look at the demo `02-vdom.html`:

```
effect(() => {  
  render(view(), app) // view() builds a VNode tree from reactive state  
})
```

`view()` reads reactive data (`people.value`), so the effect subscribed to it. Change `people` — the effect reruns, builds a new tree, and `render` diffs it and applies a targeted edit. Reactivity decides “when to repaint,” the renderer decides “how to repaint with minimal work.”

This is essentially Vue in miniature already. Only one thing is missing — being able to declare such a “state + view + effect” block as a reusable unit. That unit is called a **component**, and it’s the subject of the next layer.

Check yourself

```
npm test          # reactivity and renderer tests
npm run serve     # http://localhost:5173/playground/02-vdom.html
```

The twelve checks in `renderer.test.mjs` cover mounting, attribute updates, all children transitions, and — most importantly — keyed diff in every mode: insertion, removal, reversal, complex rearrangement. In the demo, hit “Shuffle” with the inspector open — you’ll see the `<li>` elements move rather than blink back into existence.

Components

By now we have reactivity (layer 1) and the renderer (layer 2). In the layer 2 demo we wired them together by hand with a single effect: it built an `h(...)` tree from state and called `render`. That worked, but the block couldn't be reused — state, markup, and the effect were floating free. A component packages exactly this bundle into a self-contained, reusable unit.

Chapter code: `packages/runtime-core/component.js`, `scheduler.js`, `apiLifecycle.js`, `apiInject.js`, `apiCreateApp.js`. Tests — `test/component.test.mjs`, demo — `playground/03-components.html`.

What a component is

A component is a plain object with a description: where state comes from (`setup`) and how markup is produced from it (`render`).

```
const Counter = {
  props: ['start'],
  setup(props) {
    const count = ref(props.start)
    return { count, inc: () => count.value++ }
  },
  render(ctx) {
    return h('button', { onClick: ctx.inc }, 'Clicks: ' + ctx.count)
  },
}
```

`setup` runs once when the component is created and returns whatever the markup will use. `render` is called every time the component needs to be drawn, and returns a VNode tree. Between them sits `ctx`, through which the markup reaches state. Here's how all of this comes alive.

The instance: a component's private record

The same Counter can be placed on a page ten times, and each one has its own counter. So every occurrence needs its own state. That state lives in an **instance** — an object created for each appearance of the component (`createComponentInstance`). It holds everything: parsed props, slots, the result of setup (`setupState`), the last rendered tree (`subTree`), the `isMounted` flag, the update function, and a reference to the parent.

setup and the public context

`setupComponent` prepares the instance in three steps: it parses props, lays out slots, and runs setup.

```
const setupResult = setup(instance.props, setupContext)
if (typeof setupResult === 'function') {
  instance.render = setupResult // setup returned the render function itself
} else if (setupResult && typeof setupResult === 'object') {
  instance.setupState = proxyRefs(setupResult) // the state object
}
```

Note `proxyRefs` from layer 1. It wraps the returned object so that `.value` on refs is unwrapped automatically. That's why in render you write `ctx.count`, not `ctx.count.value` — `proxyRefs` handles that.

What is `ctx`? It's `instance.ctx` — a Proxy over the instance with access rules (`PublicInstanceHandlers`). When render reads `ctx.count`, the proxy looks for `count` first in `setupState`, then in props, then among the internal `$_`-properties (`$emit`, `$slots`, `$attrs`). A single `ctx` is one access point to everything the markup can see.

One more subtlety: while setup runs, we mark the instance as “current” (`setCurrentInstance`). This lets `onMounted`, `provide`, and `inject` called inside setup know which component they belong to — without passing the instance as an argument.

Tying into reactivity: render as an effect

Here's the heart of the chapter. How do you make a component re-render itself when state changes? Wrap its rendering in a reactive effect from layer 1.

```
const componentUpdateFn = () => {
  if (!instance.isMounted) {
    const subTree = (instance.subTree = renderComponentRoot(instance))
    patch(null, subTree, container, anchor) // first time — mount
  }
}
```

```

    instance.isMounted = true
  } else {
    const nextTree = renderComponentRoot(instance)
    const prevTree = instance.subTree
    instance.subTree = nextTree
    patch(prevTree, nextTree, container, anchor) // after that – diff against the old tree
  }
}
const effect = new ReactiveEffect(componentUpdateFn, () => queueJob(instance.update))
instance.update = effect.run.bind(effect)
instance.update() // running it = mounting

```

When `componentUpdateFn` runs `render` for the first time, `render` reads reactive state — and the effect subscribes to it (exactly as in layer 1). State changes — the effect must re-run. But not immediately: its scheduler puts the component into a queue via `queueJob`. Why a queue — that’s the next section.

On the first run we mount the subtree (`patch(null, subTree, ...)`); on later runs we compare the new tree against the old one (`patch(prevTree, nextTree, ...)`), and the diff from layer 2 makes the minimal change. So reactivity answers “when to re-render,” and the renderer answers “how to re-render cheaply.”

The scheduler: three changes — one re-render

If a handler changes three reactive values in a row, a naive effect would re-render the component three times, even though the user only cares about the final result. That’s why component updates go through a queue (`scheduler.js`).

```

export function queueJob(job) {
  if (!queue.includes(job)) { // a component appears in the queue at most once
    queue.push(job)
    queueFlush()
  }
}
function queueFlush() {
  if (isFlushing) return
  isFlushing = true
  resolvedPromise.then(flushJobs) // process the queue at the end of the current tick
}

```

`resolvedPromise.then(...)` defers processing the queue to a microtask — it runs as soon as the current synchronous code finishes. By that point all changes have already happened, and the component is in the queue once — so, one re-render. That’s “batching.” The test “several changes in a row = one re-render” confirms it.

`nextTick` comes from the same place. Since the DOM update is deferred, sometimes you need to wait for it — for example, to measure the already-updated element. `await nextTick()` returns control after the queue has been processed and the DOM is current.

props: data flowing top-down

The parent passes data to a component through its VNode’s attributes: `h(Child, { label: 'hello' })`. The component declares which of those are its props with a list, `props: ['label']`. Everything declared lands in `instance.props`; everything else (for example, a `class` attached from outside) goes into `attrs`.

```
for (const key in raw) {
  if (options.has(key)) props[key] = raw[key] // a declared prop
  else attrs[key] = raw[key] // a "fallthrough" attribute
}
instance.props = reactive(props) // make props reactive
```

props are reactive, and that matters. When the parent re-renders and passes a new value, `updateComponent` calls `updateProps`, which writes the new value into the reactive props, and anyone who read that prop in `render`, `computed`, or `watch` reacts. That’s exactly what happens in the test “parent passes, child updates.”

emit: events flowing bottom-up

The reverse direction. A component doesn’t change someone else’s data directly — it “shouts up” about an event, and the parent decides what to do. The component calls `emit('increment')`, and the parent listens for it as `onIncrement`.

```
function emit(instance, event, ...args) {
  const handlerName = 'on' + event[0].toUpperCase() + event.slice(1)
  const handler = instance.vnode.props[handlerName] // onIncrement
  if (handler) handler(...args)
}
```

`emit('ping', 42)` looks in the component's props for a function `onPing` and calls it with 42. So the child reports and the parent reacts — data flows down, events bubble up. This is the basic contract for component communication.

slots: markup from outside

Sometimes the parent wants to place its own markup inside a component — like content in a card. These “holes” are called slots. A component's children are its slots:

```
h(Card, [ h('span', 'inside the card') ]) // this is the default slot
```

`normalizeSlots` turns children into an object of functions: an array/string becomes the default slot, an object `{ header, footer }` becomes named slots. The component renders them via `ctx.$slots.default()`:

```
const Card = {
  render(ctx) {
    return h('div', { class: 'card' }, ctx.$slots.default())
  },
}
```

A slot is a function (not a ready VNode) so that the content is computed at the right moment of rendering, rather than once and for all.

provide / inject: through the floors

Passing data through props is convenient between neighboring levels, but painful when you need to thread it deep down through many intermediate components. `provide` and `inject` bore a “tunnel”: an ancestor puts down a value, and any descendant pulls it out directly.

The trick is prototype-based inheritance. Each component's `provides` object inherits the parent's `provides` (`Object.create`). So reading a key naturally walks up the chain of ancestors until it finds a value. The test “provide/inject: the value reaches the grandchild” checks this across two levels of nesting.

Lifecycle

A component goes through stages: `created` → `mounted` → `updating` → `unmounted`. You can hang your own code on each one with hooks `onMounted`, `onUpdated`, `onUnmounted`, and others. A hook simply adds your function to a list on the current instance, and the renderer, when it reaches the stage, calls

the whole list. `onMounted` is handy for fetching data from the server (the element is already on the page), `onUnmounted` — for cleanup (clear a timer, unsubscribe).

createApp: the entry point

All that's left is to assemble everything into an application. `createApp(RootComponent).mount('#app')` wraps the root component in a `VNode`, attaches the application context to it (for app-level `provide` and `plugins`), and calls `render`. The `use` method registers `plugins` — this is exactly how the router and store will slot in during later layers:

```
createApp(App)
  .use(router)    // plugin: router.install(app)
  .use(pinia)
  .mount('#app')
```

Check yourself

```
npm test      # among others – 9 component tests
npm run serve # http://localhost:5173/playground/03-components.html
```

The demo is a todo list: the parent holds the state, the child `TodoItem` receives a task through `props` and sends `toggle/remove` via `emit`, and the list of components uses the keyed diff from layer 2. All of it without a single template — pure `render` and `h`. Templates (`<div>{{ x }}</div>`) are the topic of the next layer: we'll write a compiler that turns familiar markup into exactly these `h` calls.

The Template Compiler

So far we've written markup with a render function and calls to `h`. That works, but it's clunky: compare `h('li', { key: t.id }, t.text)` with the familiar `<li :key="t.id">{{ t.text }}</li>`. Templates read like HTML, which makes them clearer. The compiler is a translator: it turns a template string into exactly that render function full of `h` calls. No new runtime magic appears — just convenient syntax on top.

Chapter code: `packages/compiler/`. Tests — `test/compiler.test.mjs`, demo — `playground/04-compiler.html`.

Three translation steps

The compiler works in three passes, and this is the classic scheme of any compiler:

1. **Parsing** (`parse.js`): string → tree of objects (AST). The text `<p>{{ msg }}</p>` turns into the structure «element `p`, containing interpolation `msg`».
2. **Transformation**: turn raw attributes into meaningful directives (`v-if`, `v-for`, `:bind`, `@on`). We fold this step into generation for simplicity.
3. **Generation** (`compile.js`): walk the tree and build a string of code `h('p', null, [_s(msg)])`, then turn that string into a real function.

The output is a function indistinguishable from one you'd write by hand. The runtime from layers 1–3 knows nothing about the compiler: it doesn't care where the render function came from.

Parsing: from text to tree

A computer doesn't understand text structurally — to it, `<div>` is just five characters. The parser walks the string left to right and, recognizing familiar shapes (tag, text, `{{}}`), builds a tree of nodes. We write it as a «recursive descent»: for nested tags the function calls itself.

```
function parseChildren(context) {  
  const nodes = []
```

```

while (!isEnd(context)) {
  const s = context.source
  if (s.startsWith('{{')) nodes.push(parseInterpolation(context))
  else if (s[0] === '<' && /[a-zA-Z]/.test(s[1])) nodes.push(parseElement(context))
  else nodes.push(parseText(context))
}
return nodes
}

```

`context.source` is the remaining, not-yet-parsed part of the string. Each function «bites off» the recognized chunk from its start (`advanceBy`). `parseElement` reads the tag and attributes, then recursively calls `parseChildren` for the contents until it hits the closing `</tag>`. That’s how a tree grows out of a flat string.

Nodes come in three types: `Element` (a tag with attributes and children), `Text` (plain text), and `Interpolation` (an insertion `{{ expression }}`). The parser collects attributes as raw `{ name, value }` pairs without probing their meaning — figuring out that `v-if` is a condition and `@click` is a handler is the next step’s job.

Generation: from tree to code

Now we build a string of code from the tree. Each node type gets its own rule:

```

function genNode(node) {
  switch (node.type) {
    case 'Element':      return genElement(node)
    case 'Text':         return JSON.stringify(node.content) // "hello"
    case 'Interpolation': return `_s(${node.content})`       // _s(msg)
  }
}

```

Text becomes a string literal (`JSON.stringify` escapes quotes along the way). The insertion `{{ msg }}` becomes `_s(msg)` — a call to the helper `_s`, which coerces the value to a string. An element becomes `h('tag', props, children)`, where `props` and `children` are generated recursively from the attributes and nested nodes.

For `<div>hello</div>` you get exactly `h("div", null, ["hello"])` — what you’d write yourself. You can confirm this in the test codegen: `element with text`, or right in the demo, where the generated code for the app’s template is shown under the app.

Directives

The most interesting part is translating directives. When generating props, we classify each attribute:

- a plain `class="btn"` → a static pair `"class": "btn"`;
- `:id="bid"` (shorthand for `v-bind`) → `"id": (bid)` — the value is computed as an expression;
- `@click="inc"` (shorthand for `v-on`) → `"onClick": (inc)` — the event name gets an `on` prefix and a capital letter, and the runtime from layer 2 already knows how to attach such handlers.

Handlers have a subtlety. `@click="inc"` is a reference to a method, and we leave it as is: `(inc)`. But `@click="count++"` is an action expression, and it must be wrapped, otherwise `count++` runs once at generation time rather than on click:

```
function genHandler(exp) {
  const isMethodPath = /^[A-Za-z_$][\w$.]*$/ .test(exp.trim())
  return isMethodPath ? `(${exp})` : `$event => (${exp})`
}
```

`v-if` and `v-for` are «structural» directives: they control whether nodes appear at all, not their attributes. `v-for="item in list"` wraps the node in the helper `_l` (render list), which iterates over the list and returns an array of nodes:

```
// <li v-for="item in items">{{ item }}</li> turns into:
_l(items, (item) => h("li", null, [_s(item)]))
```

`v-if` is generated at the level of the children list, because it needs access to a neighboring `v-else`. The chain `v-if` / `v-else-if` / `v-else` is assembled into a ternary operator:

```
// <span v-if="ok">yes</span><span v-else>no</span> →
(ok) ? h("span", null, ["yes"]) : h("span", null, ["no"])
```

With no `v-else`, the branch becomes `: null`, meaning «render nothing».

From string to function: with(ctx)

The final trick is turning the generated string into a real function. Here a question comes up: in the code `h("p", null, [_s(msg)])`, where does `msg` come from? It has to come from the component's state, from `ctx`. Dragging `ctx` in front of every name in the generator is a chore. Instead, we wrap the body in `with(ctx)`:

```
new Function('h', 'Fragment', '_s', '_l',
  `return function render(ctx){ with(ctx){ return ${code} } }`)
```

`with(ctx)` forces every name inside to be looked up in `ctx` first. So `msg` resolves as `ctx.msg`, and `count` as `ctx.count`, without a single `ctx.` in the code. The helpers `h`, `_s`, `_l` come in as parameters of the outer factory function, bypassing `ctx`.

`with` is a rare and usually undesirable JavaScript operator, but here it fits, and Vue's real runtime compiler uses the same trick (`with(this)`). For `with` to decide correctly which name comes from `ctx` and which comes from the outer scope, we added a `has` trap to the component context proxy (layer 3): it answers «yes» for state and props, and «no» for everything else.

How the compiler hooks into the runtime

The runtime doesn't depend on the compiler directly — otherwise you couldn't build a «lightweight» version without it. The link is established through registration: importing `packages/compiler/index.js` calls `registerRuntimeCompiler(compile)`, and from that point on a component with a `template` property is compiled automatically at initialization (`finishComponentSetup` in layer 3). The full build `packages/minivue.js` does this import for you — the way the main `vue` package bundles both the runtime and the compiler.

What we simplified

Vue's real compiler does far more: static analysis and hoisting of unchanging nodes, patch flags to speed up diffing, handler caching, `v-model`, slots, event modifiers (`@click.stop`), compilation in a separate build step. We took the core idea — parsing, generation, `with(ctx)` — and the most needed directives. That's enough to understand how `<div>{{ x }}</div>` becomes a working interface, and to write real applications with templates.

Check yourself

```
npm test      # among others – 10 compiler tests
npm run serve # http://localhost:5173/playground/04-compiler.html
```

The tests check both the generated code (`compileToString`) and the full cycle «template → DOM»: interpolation, reactive updates, `@click`, `v-if/v-else`, `v-for`. In the demo the todo list is now written as a template, and at the bottom of the page you can see the code the compiler turned it into. Next up — the router: we'll teach the app to show different pages based on the browser's address.

The Router

A regular website asks the server for a fresh page on every navigation and reloads it. A single-page application (SPA) doesn't work that way: it loads once, then swaps its own content by looking at the URL in the browser's address bar. The router handles that. Ours is a scaled-down copy of Vue Router, and it rests entirely on the reactivity from layer 1.

Chapter code: `packages/router/`. Tests — `test/router.test.mjs`, demo — `playground/05-router.html`.

The idea: URL → component

The router solves one problem: pick which component to show based on the current URL. `/` — the Home component, `/about` — About, `/user/42` — User with the parameter `id = 42`. The chosen component is shown in a special place — `<RouterView>`. `<RouterLink>` links change the URL without reloading the page.

The whole trick is that the “current route” is a reactive object. `<RouterView>` reads it in its render function, so by the rules of layer 1 it subscribes to it. The URL changes — the route changes — `<RouterView>` re-renders with the new component. The router needs no special reactivity mechanism of its own; it reuses ours.

history: where the URL lives

The URL can be stored in different ways, depending on the environment. So the mechanism is hidden behind a `history` object with a single interface: `location` (the current path), `push` and `replace` (change it), `listen` (subscribe to changes). There are three implementations:

- **`createWebHistory`** — normal URLs like `/about` via the History API (`pushState`). Clean, but the server must serve `index.html` for any path.
- **`createWebHashHistory`** — URLs with a hash, `/#/about`. The part after `#` never reaches the server, so it works on any static host with no configuration.

- **createMemoryHistory** — the URL lives in a plain variable, with no window. Needed for tests and for the server (layer 7), where there's no browser.

The router doesn't care which one is plugged in — it talks through the same interface. Tests use `createMemoryHistory`, the demo uses `createWebHashHistory`.

The matcher: parsing paths with parameters

The route `/user/:id` should match the URL `/user/42` and produce `id = 42`. To do that, we turn the route's path into a regular expression, and the `:parameters` into capture groups:

```
function compileRoute(record) {
  const keys = []
  const pattern = record.path
    .replace(/\\/g, '\\\\')
    .replace(/:(\w+)/g, (_, name) => {
      keys.push(name)      // remember the parameter name
      return '([^/]+)'     // and replace :id with "any segment without a slash"
    })
  return { ...record, regex: new RegExp('^' + pattern + '$'), keys }
}
```

`resolve(path)` iterates over the routes, applies their regexes, and for the first match collects params from the capture groups using the saved names. If nothing matches, an empty route is returned, and `<RouterView>` shows nothing.

The reactive current route

The heart of the router is the reactive object `currentRoute`:

```
const currentRoute = reactive({ path: '/', params: {}, matched: [] })

function applyRoute(path) {
  const r = resolve(path)
  currentRoute.path = r.path
  currentRoute.params = r.params
  currentRoute.matched = r.matched // the record(s) of the matching route
}
```

`applyRoute` writes the parse result into the reactive object — and that's the only thing needed to update the UI. Everything that read `currentRoute` reacts. The router subscribes to the history

(`history.listen(applyRoute)`), so both our push and the browser's back/forward buttons lead to the same thing — a re-render.

Navigation and guards

push doesn't change the URL right away — first it runs the navigation hooks (`beforeEach`). A hook receives where we're going and where we're coming from, and it can allow the transition, cancel it (by returning `false`), or redirect (by returning a different path):

```
function navigate(to, replace) {
  const toRoute = resolve(typeof to === 'string' ? to : to.path)
  for (const guard of guards) {
    const result = guard(toRoute, currentRoute)
    if (result === false) return // cancel
    if (typeof result === 'string') return navigate(result, replace) // redirect
  }
  history[replace ? 'replace' : 'push'](toRoute.path) // and only now change the URL
}
```

This is how you protect pages: “don't let an unauthenticated user into `/admin`, send them to `/login`”. The tests cover both branches — cancellation and redirect.

RouterView and RouterLink

`<RouterView>` is a component of a few lines. It injects the current route and in render returns the component from matched:

```
const RouterView = {
  setup() {
    const route = inject(ROUTE_KEY)
    return () => {
      const matched = route.matched[0]
      return matched ? h(matched.component) : null
    }
  },
}
```

The key point is `return () => ...`. We return a render function from `setup`, and it reads `route.matched` on every call. So the component's effect is subscribed to the route, and on navigation `<RouterView>` re-renders itself.

`<RouterLink>` renders an ordinary `<a>`, but it intercepts the click: it cancels the browser's default navigation (`preventDefault`) and calls `router.push` — so the URL changes without a reload.

Wiring into the app

The router is a plugin. `app.use(router)` calls its `install`, where the router hands itself and the route out through `provide`, registers `<RouterView>` / `<RouterLink>` as global components, and puts `$router` / `$route` into the global properties:

```
install(app) {  
  app.provide(ROUTER_KEY, router)  
  app.provide(ROUTE_KEY, currentRoute)  
  app.component('RouterView', RouterView)  
  app.component('RouterLink', RouterLink)  
  app.config.globalProperties.$router = router  
  app.config.globalProperties.$route = currentRoute  
}
```

For `<RouterView>` in a template to turn into a component rather than an unknown HTML tag, the compiler (layer 4) got one extra ability: tags starting with an uppercase letter or containing a hyphen are generated via `_c('RouterView')` — resolving a component by name among the registered ones. Inside setup, the same router and route are reached with the `useRouter()` and `useRoute()` hooks.

What we simplified

The real Vue Router does far more: nested routes (several `<RouterView>` deep), named routes and views, lazy-loaded components, async guards with `next()`, query parameters and query-string parsing, scroll behavior, meta fields, and transition animations. We took the skeleton — history, a matcher with parameters, a reactive route, `RouterView/RouterLink`, `beforeEach` — which is enough to understand the principle and build a working multi-page application.

Check yourself

```
npm test      # among other things – 6 router tests  
npm run serve # http://localhost:5173/playground/05-router.html
```

The tests run navigation against the in-memory history: the starting route, `push`, `:id` parameters, an empty route, and both kinds of guards. The demo is an app with a menu of `RouterLinks` and a

user page that reads `:id`. Next up — the store: shared application state available to any component without passing it through props.

The Store

A component's state lives inside it, and that's correct: a button's counter shouldn't leak outside. But part of the state is shared across the whole application — the cart, the current user, the color theme. Threading it through props and events across the entire component tree is tedious and fragile. The store is a separate reactive storage that any component talks to directly. Our store is a scaled-down copy of Pinia.

Chapter code: `packages/store/`. Tests — `test/store.test.mjs`, demo — `playground/06-store.html`.

The store is reactivity again

The core idea: the store has no new mechanism. Its state is reactive/ref from layer 1, its computed fields are computed, its actions are plain functions that mutate state. A component that reads the store in render subscribes to it exactly as it does to its own ref. When the store changes, every component that read it re-renders, wherever in the tree it sits. The store just gives this shared reactive state a name and convenient access.

createPinia: the store container

`createPinia()` creates a container that plugs into the application as a plugin (`app.use(createPinia())`). Inside it holds a map of ready stores, shared reactive state, and a list of plugins:

```
export function createPinia() {
  const pinia = {
    _stores: new Map(),      // id → ready store (created lazily, once)
    _plugins: [],
    state: reactive({}),    // state[id] = state of store id
    install(app) {
      setActivePinia(pinia)
      app.provide(PINIA_KEY, pinia)
    },
  },
```

```
    use(plugin) { pinia._plugins.push(plugin); return pinia },  
  }  
  return pinia  
}
```

install records the container as the “active” one and hands it out through provide — so any component can reach it.

defineStore: two styles

defineStore(id, ...) declares a store. As in Vue, two styles are supported — Options and Setup — and both collapse to the same thing inside.

Options — familiar and declarative:

```
const useCounter = defineStore('counter', {  
  state: () => ({ count: 0 }),  
  getters: { double: (s) => s.count * 2 },  
  actions: { inc() { this.count++ } },  
})
```

Setup — flexible, built on the Composition API:

```
const useCart = defineStore('cart', () => {  
  const items = ref([])  
  const total = computed(() => items.value.reduce((s, p) => s + p.price, 0))  
  const add = (p) => (items.value = [...items.value, p])  
  return { items, total, add }  
})
```

defineStore returns not the store itself but a useStore() function. On the first call it creates the store; on later calls it hands back the same instance. The store is a singleton per application:

```
function useStore() {  
  const pinia = getActivePinia()  
  if (!pinia._stores.has(id)) createStore(id, setupOrOptions, pinia)  
  return pinia._stores.get(id)  
}
```

The “store is a singleton” test checks this: two calls to useCounter() return the same object, and a change through one is visible in the other.

One creation path

To avoid writing the logic twice, we turn an options store into a setup function (`optionsToSetup`) — and from there both styles follow the same road:

```
function setup() {
  const state = reactive(options.state ? options.state() : {})
  pinia.state[id] = state
  const parts = {}
  for (const key in state) parts[key] = toRef(state, key)           // state → refs
  for (const name in options.getters) parts[name] = computed(...) // getters → computed
  for (const name in options.actions) parts[name] = (...a) => ... // actions → functions
  return parts
}
```

The result is a `parts` object of refs, computeds, and functions. All that's left is to make it ergonomic.

proxyRefs: why `store.count`, not `store.count.value`

`parts` holds refs, which means from the outside you'd have to write `store.count.value`. Ugly. `proxyRefs` from layer 1 saves us — it wraps the object so that on read a ref unwraps automatically, and on write the value goes into `.value`:

```
store = proxyRefs(parts)
```

Now `store.count` returns a number, `store.count = 5` writes to state, and `store.inc()` calls the function. `proxyRefs` leaves functions alone — they pass through as is. One line turns a bag of refs into an ergonomic store.

Inside options actions this points at the store, so `this.count++` reads and writes state through the same `proxyRefs`, staying reactive. The “effect catches changes” test confirms it: a subscription to `store.count` fires on every `inc()`.

storeToRefs: destructuring without losing reactivity

It's tempting to write `const { count, double } = store`, but that loses reactivity — `count` becomes a plain number at the moment of destructuring. For state and getters there's `storeToRefs`: it hands them back as refs that keep their link to the store. Actions you take straight from the store (they're safe to destructure — they're functions):

```
const { count, double } = storeToRefs(store) // reactive refs
const { inc } = store                        // actions – as is
```

This works through `toRef(store, key)` from layer 1: the resulting ref reads and writes `store[key]`, which means it stays part of the reactive system.

Plugins

`pinia.use(plugin)` adds an extension that runs when each store is created and receives `{ store, pinia, id }`. Whatever the plugin returns is mixed into the store. That’s how cross-cutting features are built: persisting a store to `localStorage`, logging actions, adding shared fields. For us it’s a few lines in `createStore`.

What we simplified

Real Pinia does more: `$patch` for batched changes, `$subscribe` on state changes and `$onAction` on actions, `$reset`, hot module replacement during development, devtools integration, and type safety through TypeScript. We took the essence — reactive singleton state, getters on computed, actions, both declaration styles, `storeToRefs`, and plugins. That’s enough to grasp the idea and share state between any components.

Check yourself

```
npm test      # among others – 7 store tests
npm run serve # http://localhost:5173/playground/06-store.html
```

The tests cover both styles, reactivity of state and getters, singleton behavior, `storeToRefs`, and the store working inside a component. The demo is a mini-shop: the “header” and the “catalog” are two independent components, but they share the cart through the store, without a single prop between them. One last layer remains — SSR: we’ll learn to render the application to HTML on the server and “hydrate” it in the browser.

SSR and Hydration

Until now the app was born in the browser: JavaScript loaded, it built the DOM, and only then did the user see any content. That has two problems. First, a blank screen while the code loads and runs. Second, search crawlers often see an empty page. The fix is to draw the app ahead of time, on the server, and ship ready-made HTML. That is server-side rendering (SSR). And to make the shipped HTML come alive and become interactive, the client “wakes it up” — that is hydration. Both mechanisms are the topic of this final layer.

Code for this chapter: `packages/server-renderer/` (the server) and the `hydrate` function in `packages/runtime-core/renderer.js` (the client). Tests are in `test/ssr.test.mjs`; the live example is `playground/07-ssr/` (a Node server).

Rendering to a string

The key observation: a VNode tree doesn’t have to turn into DOM specifically. It’s just as easy to turn it into text — an HTML string. No browser is needed for that, only a walk over the tree and some string concatenation. That’s the job of `renderToString`:

```
function renderVNode(vnode, parentComponent) {
  const { type } = vnode
  if (type === Text) return escapeHtml(vnode.children)
  if (type === Fragment) return renderChildren(vnode.children, parentComponent)
  if (typeof type === 'string') return renderElement(vnode, parentComponent)
  // component:
  const { instance, subTree } = createSSRComponent(vnode, parentComponent)
  return renderVNode(subTree, instance)
}
```

A text node becomes escaped text, an element becomes the string `<tag attributes>children</tag>`, a component becomes the result of its render. For a component on the server we create an instance and run setup (`createSSRComponent`), but we don’t set up a reactive effect: the server has nothing to re-render, it needs a single snapshot.

Two details matter for correct HTML. First, **event handlers aren't serialized**: `onClick` makes no sense in HTML, the handler will be attached on the client. Second, **escaping**: user data goes through `escapeHtml`, otherwise the string `<script>` inside text would turn into executable code (XSS). The test “escaping against XSS” checks exactly this.

createSSRApp

Just as the browser has `createApp`, the server has `createSSRApp`: the same interface with `use`, `provide`, `component`, but instead of `mount` there's a `renderToString` method that returns HTML. Plugins (router, store) are wired up the same way through `use`, so server rendering of a full app looks almost like the client version — that's the point of “isomorphism”: one codebase for two environments.

The whole server

For every page request the server (`playground/07-ssr/server.js`) does three things:

```
const appHtml = renderToString(createVNode(App)) // 1. draw the app into a string
res.end(page(appHtml))                          // 2. embed it into the HTML shell
// 3. the shell contains the <script> for client hydration
```

The shell is a plain HTML page where `<div id="app">` already holds the finished markup, followed by the client script. The user sees the content instantly, before a single line of JavaScript runs. Open “View Source” and you'll see the counter already drawn there — that's SSR.

Notice: App is the very same module for server and client. The server imports it in Node, the client imports it in the browser. This is an isomorphic app.

The problem: HTML without life

The HTML the server ships is static. Buttons are drawn but don't react to clicks — they have no handlers (we didn't serialize them). The naive fix is to just run a normal `mount` on the client. But that would tear down all the server HTML and create an identical copy from scratch: wasted work, flicker, lost state (cursor position in an input, scroll offset). The right approach is not to create but to “adopt” the nodes that are already there. That's hydration.

hydrate: adopting the existing DOM

hydrate walks the VNode tree in parallel with the existing DOM. For each node it doesn't create a new one — it links the VNode to the real node it finds (`vnode.el = node`) and adds whatever the HTML lacks, above all event handlers:

```
function hydrateNode(node, vnode) {
  vnode = normalizeVNode(vnode)
  const { type } = vnode
  if (type === Text) { vnode.el = node; return node.nextSibling }
  if (typeof type === 'string') {
    vnode.el = node // adopt the existing element
    for (const key in vnode.props) {
      if (key !== 'key') hostPatchProp(node, key, null, vnode.props[key]) // attach events
    }
    let cur = node.firstChild
    for (const child of vnode.children) cur = hydrateNode(cur, child) // recurse into children
    return node.nextSibling
  }
  // component — adopted "on top of" the node, with a reactive effect set up
  hydrateComponentImpl(vnode, node)
  return node ? node.nextSibling : null
}
```

For a component, hydration does the same thing as mounting, except the first render pass doesn't create the subtree through `patch(null, ...)` — it adopts the existing DOM through `hydrateNode`. A normal reactive effect is set up in the process — so from then on the component lives as usual: state changes, `patch` runs, but now over the adopted nodes.

The test “hydrate adopts the DOM, attaches events, and updates” checks all of this strictly: the server-side button node and the client-side one after hydration are the same object (`Object.is`), meaning the node wasn't recreated; a handler appeared on it; and a click changes state and patches that same button. That's exactly how, after hydration, the static server HTML becomes a full-fledged SPA without a single needless recreation.

What we simplified

Real SSR in Vue is more involved in the details: it inserts comment nodes as anchors for fragments and portals, carefully compares the server and client output and complains about discrepancies (hydration mismatch), serializes and transfers store state to the client (`window.__INITIAL_STATE__`), sup-

ports streaming rendering, async setup and Suspense, and fragment caching. We built the skeleton — rendering a tree to a string with escaping, an isomorphic server, and hydration that adopts nodes and attaches events — which is enough to understand how SSR works and to build a working example.

Check yourself

```
npm test # among others – 7 tests for SSR and hydration
node playground/07-ssr/server.js # then open http://localhost:5174/
```

The tests cover `renderToString` (attributes, absence of events, escaping, nested components, void tags, `createSSRApp`) and the full hydration cycle. In the Node example the server ships ready-made HTML and the client wakes it up — open the page source to see the server markup, and click the buttons to confirm it came alive.

Wrap-up

We’ve gone through all seven layers and built a working analog of Vue from scratch: reactivity, a virtual DOM with fine-grained diffing, components, a template compiler, a router, a store, and SSR. None of these mechanisms should be a black box to you anymore — you wrote each one yourself. Now, when you open the real Vue, you’ll see not magic but familiar ideas, just polished to production quality. And that was the goal all along: to understand how it works, from the inside.

Forms and Bindings

The core of the framework is done, but the first real form runs into three annoyances. An input field has to be both read and written by hand. Classes want to be toggled by condition, not glued together as a string. And an event handler constantly needs `preventDefault` and a check for which key was pressed. Vue covers this with three conveniences: `v-model`, object bindings for `class/style`, and event modifiers. All three are layers on top of what already exists, and they cost little to add.

Chapter code: `packages/shared.js` (class/style normalization), changes in `packages/compiler/compile.js` (`v-model`, modifiers) and `packages/runtime-dom/patchProp.js`. Tests — `test/forms.test.mjs`, demo — `playground/08-forms.html`.

class and style bindings

In layer 2, `class` could only be a string. An object (“`class` → is it on?”) or an array is more convenient:

```
<span :class="{ active: isActive, done: isDone }" />
<span :class="['btn', isPrimary && 'btn-primary']" />
<div :style="{ color: 'red', fontSize: size + 'px' }" />
```

So that both the browser and SSR read every form the same way, we reduce it to a canonical shape in `shared.js`. `normalizeClass` turns an object/array into a string, `normalizeStyle` into an object, and `styleToString` into a string with kebab-case properties:

```
normalizeClass({ active: true, off: false }) // 'active'
normalizeClass(['a', false, 'b'])           // 'a b'
styleToString({ color: 'red', fontSize: '14px' }) // 'color:red;font-size:14px'
```

These helpers are called in two places: in the browser `patchProp` (when setting `class/style` on a real element) and in the server-side `renderAttrs`. One piece of logic for both paths. `fontSize` becomes `font-size` automatically, so a style object can use the familiar JS property names.

v-model: two-way binding

An input field is a two-way street: state should show up in the field, and the user’s input should update state. By hand that’s two things at once:

```
<input :value="name" @input="name = $event.target.value" />
```

v-model is sugar over exactly this pair. The compiler expands `<input v-model="name">` into a value binding plus a handler. Look at the implementation in `applyVModel`: it picks the property and the event based on the field type, because fields differ:

- text input and textarea → `:value + @input`;
- checkbox → `:checked + @change` (and we read `$event.target.checked`);
- select → `:value + @change`.

```
directives.binds.push({ arg: prop, exp }) // :value="name"
directives.ons.push({ event, exp: `${exp} = $event.target.${field}` }) // @input="name = ..."
```

From there it goes down the normal props-generation path — no special node. Models support modifiers too: `.number` wraps the value in `Number(...)`, `.trim` in `.trim()`. The “input updates state” test checks both sides: that state shows up in the field, and that input changes it.

A subtlety on the runtime side: for value and checked, `patchProp` writes to the element property (`el.value`, `el.checked`), not the attribute. The attribute only sets the initial value; an already-rendered field can be updated only through the property — without this, v-model would get “stuck”.

Event modifiers

Handlers almost always start with boilerplate: “don’t reload the page”, “don’t bubble up”, “only react to Enter”. Vue moves these into modifiers after the event name:

```
<form @submit.prevent="save" />      <!-- $event.preventDefault() -->
<div @click.stop="onClick" />        <!-- $event.stopPropagation() -->
<input @keyup.enter="submit" />      <!-- only when Enter is pressed -->
```

The compiler parses `@submit.prevent` into an event plus a list of modifiers (`parseEvent`), and `genHandler` builds a wrapper with the needed guards:

```
function genHandler(on) {
  if (on.modifiers.length === 0) {
    // no modifiers – same as before: method reference or inline expression
    return isMethodPath ? `(${exp})` : `$event => (${exp})`
  }
}
```

```
}  
const guards = []  
if (keyMods.length) guards.push(`if(!_key($event,${JSON.stringify(keyMods)}))return;`)  
if (mods.includes('stop')) guards.push('$event.stopPropagation();')  
if (mods.includes('prevent')) guards.push('$event.preventDefault();')  
if (mods.includes('self')) guards.push('if($event.target!==$event.currentTarget)return;')  
return `$event => { ${guards.join('')} ${isMethodPath ? `${exp}($event)` : `${exp}}` }`  
}
```

Key modifiers (`.enter`, `.esc`, `.up`, and others) are checked through the `_key` helper, which maps a modifier name to an `event.key` value (`enter` → `'Enter'`). The tests check both the generated code and the behavior: `@click.prevent` really does call `preventDefault`, and `@keyup.enter` fires only on `Enter`.

The “no modifiers” path stays as it was — a short reference (`inc`) or `$event => (count++)`. That way the old code and tests from layer 4 keep working unchanged, and the wrapper appears only where there actually are modifiers.

What we simplified

Real Vue supports more: `v-model` on components (with `modelValue` and `update:modelValue`), multiple models on one component, `.lazy`, `v-model` on radio and multiple select, system key modifiers (`.ctrl`, `.shift`), `.once`, `.capture`, `.passive`. We took the most common cases — text fields, checkbox and select, object/array class/style, `.stop/.prevent/.self`, and keys. That’s enough to build a full-fledged form, and the principle behind the rest is exactly the same.

Check yourself

```
npm test      # among others – 11 tests for forms and bindings  
npm run serve # http://localhost:5173/playground/08-forms.html
```

The demo has a form with a text field, a select, and a checkbox on `v-model`, a badge whose class is toggled by a `:class` object, and submission via `@submit.prevent` and `@keyup.enter`. The “live state” under the form always matches the fields — that’s two-way binding in action.

Reactivity, Deeper

The reactivity core from layer 1 covers 90% of what you need, but real code brings up a few more tools. They're small and built entirely on the `effect`, `track`, and `trigger` we already have. This chapter is short — we're just picking up some convenient primitives.

Chapter code: `packages/reactivity/` (additions in `reactive.js`, `ref.js`, `watch.js`). Tests — `test/reactivity-extras.test.mjs`.

watchEffect

`watch` requires an explicit source and hands you the old and new value. Often that's overkill — you just want “run this and re-run it whenever something changes.” That's what `watchEffect` is for:

```
const stop = watchEffect(() => {
  console.log('count =', count.value) // reads count → subscribes to it
})
```

It runs the function immediately (collecting dependencies from whatever it read along the way), then re-runs it when those change. The implementation is literally an effect whose scheduler re-runs it:

```
export function watchEffect(fn) {
  const effect = new ReactiveEffect(fn, () => effect.run())
  effect.run()
  return () => effect.stop()
}
```

It returns a stop function — call it and the watching stops (`effect.stop` runs cleanup and marks the effect inactive, see layer 1).

readonly

Sometimes you want to hand out data “view only” — for example, a provide value that descendants shouldn’t mutate. `readonly(obj)` returns a Proxy where reads work (including nested ones — those become `readonly` too), while writes are silently rejected with a warning:

```
export function readonly(target) {
  return new Proxy(target, {
    get(obj, key, receiver) {
      if (key === RAW) return obj
      const result = Reflect.get(obj, key, receiver)
      return isObject(result) ? readonly(result) : result // nested is readonly too
    },
    set(obj, key) {
      console.warn(`readonly: cannot modify "${String(key)}"`)
      return true // the write is "swallowed" but not performed
    },
  })
}
```

There’s nothing to track here — the value never changes, so track in get isn’t needed.

shallowReactive and shallowRef

Deep reactivity is handy, but sometimes excessive. If you only ever change a large object by replacing it wholesale (rather than mutating its inner fields), there’s no point wrapping every level. The “shallow” versions react only to the top level:

- `shallowReactive(obj)` — `state.count` is reactive, but `state.nested.x` isn’t. In get we simply don’t wrap nested objects.
- `shallowRef(v)` — reacts to replacing `.value` wholesale, but not to changing a field inside. If you did change a field and want to notify subscribers manually — there’s `triggerRef(ref)`.

The test “`shallowRef` reacts to replacement but not to mutation” shows the difference: `s.value.count = 5` doesn’t wake the effect, while `s.value = { count: 10 }` does.

markRaw

The opposite problem: mark an object so it **never** becomes reactive. You do this with heavy third-party objects (a map instance, a class from a library) that don’t need reactivity and are even hurt by it performance-wise:

```
const map = markRaw(new ExpensiveThing())  
const state = reactive({ map }) // state.map stays a plain object
```

The implementation is an invisible `Symbol` marker on the object that `reactive` checks before wrapping anything nested: see the marker, return it as is.

Wrap-up

All five additions are thin layers over the layer-1 machinery, with no new “engine.” That’s a good test of how clean the reactivity turned out: if new tools take just a few lines each to write, the foundation is right. Next up — directives and dynamic components, where we’ll get real new mechanics again.

Directives and Dynamic Components

Three features that real apps rarely do without: your own directives (`v-focus`, `v-tooltip`), swapping a component based on data (`<component :is>`), and `v-model` on your own components. All three build on the component and compiler machinery we already have.

Chapter code: changes in `packages/compiler/compile.js`, `runtime-core/renderer.js`, `runtime-core/vnode.js`, `runtime-core/component.js`. Tests — `test/directives.test.mjs`, demo — `playground/10-directives.html`.

Custom directives

A directive is a way to attach low-level behavior to a plain element: set focus, wire up a third-party library, watch when the element appears on screen. A directive is an object with hooks tied to the element's lifecycle:

```
app.directive('focus', {  
  mounted(el) { el.focus() }, // element entered the DOM — focus it  
})
```

Hooks receive the real element `el` and a binding object with usage data: `value` (the current value of the expression), `oldValue` (the previous one, in `updated`), `arg` (the argument after the colon), and `modifiers`. So `v-color:bg.important="c"` gives the hook `binding.value = c`, `binding.arg = 'bg'`, `binding.modifiers = { important: true }`.

How it works

When the compiler meets an unknown `v-*`, it puts it in a separate list (`parseDirective`) and wraps the element in a `_wd` (`withDirectives`) call, passing a tuple of “directive, value, argument, modifiers” for each directive:

```
// <div v-focus> compiles to:  
_wd(h("div", null, [...]), [[_dir("focus"), void 0, void 0, {}]])
```


`_dir` (`resolveDirective`) finds the directive by name — first in the component’s local `directives` option, then among the global `app.directive` registrations. `withDirectives` just attaches the parsed bindings to `vnode.dirs`.

The renderer takes over from there. After inserting the element it calls the `mounted` hook, after an update — `updated` (having first stashed the old value into `oldValue`), and before removal — `beforeUnmount` and `unmounted`:

```
function invokeDirectives(vnode, name) {
  for (const binding of vnode.dirs || []) {
    const hook = binding.dir[name]
    if (hook) hook(vnode.el, binding, vnode)
  }
}
```

Directives are just extra calls at the same points in the element’s lifecycle where the renderer was already doing work. The “mounted/updated/unmounted” test checks all three moments and that binding is correct.

Dynamic components

Sometimes the data decides which component to show: tabs, wizard steps, a list of heterogeneous blocks. For this there’s a special tag `<component :is="...">`, where `:is` is an expression holding a component (an object) or its name (a string):

```
<component :is="currentTab" />
```

The compiler recognizes the `component` tag, takes its `:is`, and generates the node type through the `_cd` helper instead of a string tag:

```
const _cd = (is) => (typeof is === 'string' ? resolveComponent(is) : is)
// <component :is="cur"> → h(_cd(cur), null, [...])
```

If `is` is a string, `_cd` finds the component by name; if it’s an object, it uses it directly. From there it’s the same as any component: `change` `is` and `patch` sees a different type, unmounts the old component, and mounts the new one.

A practical subtlety shows up in the demo and the test: the component goes into a `shallowRef`, not a plain `ref`. Otherwise `ref` would try to make the component definition reactive (wrap it in a `Proxy`), which is pointless and harmful. The `shallowRef` from layer 9 stores the value as-is — exactly what these cases need.

v-model on components

We already covered v-model for input fields. On a component it works on the same “value down, event up” principle, only through the modelValue prop and the update:modelValue event:

```
<MoneyInput v-model="amount" />

```

Built-in Components

Vue ships a few “system” components that don’t behave like ordinary ones: Teleport renders its children somewhere else on the page, KeepAlive keeps inactive components alive, and defineAsyncComponent loads code on demand. They’re special — the renderer recognizes them by markers and handles them differently.

Chapter code: `packages/runtime-core/builtins.js` plus changes in `renderer.js`. Tests — `test/builtins.test.mjs`, demo — `playground/11-builtins.html`.

Teleport: render somewhere else

A modal logically belongs to the component that opened it, but physically it should live at the end of `<body>` — otherwise a parent’s overflow: hidden clips it or someone else’s z-index buries it. Teleport breaks that tie: in the markup the children sit inside the component, but in the DOM they land in the target container.

```
<Teleport to="#modals">
  <div class="modal">...</div>
</Teleport>
```

The component itself is just a marker — `{ __isTeleport: true }`. All the work happens in the renderer, which, on seeing this marker, sends the children to the target container instead of the current one:

```
function processTeleport(n1, n2, container, anchor) {
  const target = resolveTeleportTarget(n2.props) // element or querySelector(to)
  if (n1 == null) {
    n2.el = hostCreateText('') // empty anchor at the original spot
    hostInsert(n2.el, container, anchor)
    mountChildren(n2.children, target, null) // children go to target
  } else {
    patchChildren(n1, n2, n1.target, null) // updates go to target too
  }
}
```

```
}
}
```

An empty text anchor stays at the original spot so sibling nodes don't get thrown off, while the content lives in the target container. The “children render in the target container” test checks exactly this: the original spot is empty, all the content is in the target.

KeepAlive: hide, don't destroy

Normally, switching a dynamic component destroys the old one along with all its state: type text into a form, move to another tab, come back — the field is empty. KeepAlive fixes that: an inactive component isn't destroyed, it's hidden, keeping its instance and state.

```
<KeepAlive><component :is="currentTab" /></KeepAlive>
```

The implementation is a combination of “markers on the vnode + special handling in the renderer.” KeepAlive itself keeps a cache of “key → vnode with a live instance” and sets two markers:

```
if (cache.has(key)) {
  vnode.component = cache.get(key).component // reuse the live instance
  vnode.__keptAlive = true                    // renderer will "revive", not mount
} else {
  cache.set(key, vnode)
}
vnode.__shouldKeepAlive = true                // on leave - hide, don't destroy
```

The renderer reacts to the markers in two places. On unmount, instead of removing the component, it hides its DOM in off-DOM storage without touching the instance:

```
if (vnode.__shouldKeepAlive) {
  hostInsert(vnode.component.subTree.el, keepAliveStorage()) // hidden away
  return
}
```

And when “mounting” a cached component, it doesn't create it anew — it brings the hidden DOM back:

```
if (n1 == null && n2.__keptAlive) {
  hostInsert(n2.component.subTree.el, container, anchor) // pulled from storage
  n2.el = n2.component.subTree.el
}
```

The instance stays alive the whole time, its reactive effect is never stopped — which is why the state is right where you left it. The “state is preserved across switching” test proves it: a counter incremented on tab A stays put after moving to B and back, instead of resetting.

A slots bug found along the way

KeepAlive exposed a subtle bug. Its setup returns `() => slots.default()`, capturing `slots` in a closure. But our code, when updating a component, **reassigned** `instance.slots` to a new object — and the closure kept seeing the old one. Because of this, KeepAlive always showed the first tab. The fix: update the contents of the same `slots` object instead of replacing the reference (`updateSlots` in `component.js`). A good lesson: reference stability matters anywhere something gets captured in a closure.

Async components

There’s no reason to load a large app all at once — rarely used screens can be pulled in later. `defineAsyncComponent` wraps a loader (usually a dynamic import) in a component that shows “loading” while the code is on its way, then swaps in the real component:

```
const Chart = defineAsyncComponent(() => import('./Chart.js'))
```

The implementation is, again, just reactivity. The wrapper’s setup kicks off the loader and holds a state ref; when the promise resolves, the ref flips — and the wrapper re-renders with the real component:

```
setup() {
  const loaded = ref(false)
  options.loader().then((mod) => {
    resolvedComponent = mod.default || mod
    loaded.value = true // flipping the ref → re-render the wrapper
  })
  return () =>
    loaded.value && resolvedComponent
      ? h(resolvedComponent)
      : h('span', 'Loading...')
}
```

No special machinery — reactivity switches the view on its own. The tests cover both outcomes: a successful load and an error (in which case `errorComponent` is shown).

What we simplified

We left out `Suspense` — the coordinator for several async dependencies with a shared fallback and async setup. It hooks into deeper promise handling inside the render and would deserve a whole chapter of its own; for teaching purposes `defineAsyncComponent` is enough to show the essence of “loading → ready.” Also, in real Vue `KeepAlive` supports `include/exclude`, a cache limit (`max`), and the `activated/deactivated` hooks, while `Teleport` has disabled. We took the core of each.

Check yourself

```
npm test          # among others – 5 tests for the built-in components
npm run serve     # http://localhost:5173/playground/11-builtins.html
```

In the demo: a modal moves into `#modals` via `Teleport`, tabs under `KeepAlive` retain the text you typed, and the async block appears after a delay. One last layer remains — store extensions and the big final app, where everything comes together.

Store Deep-Dive and the Capstone App

The last layer rounds out the store's conveniences and brings everything we've built together into one living app. There's almost no new deep machinery here — this is where the picture gets completed.

Chapter code: additions in `packages/store/index.js`, the final app in `playground/12-capstone.html`.

\$patch, \$subscribe, \$reset

Three utility methods on the store that you reach for constantly in real work.

\$patch changes several fields at once — with an object or a function. An object is handy for straight replacements, a function for computing from the current state:

```
store.$patch({ count: 5, name: 'y' })      // merge an object
store.$patch((s) => { s.count++; s.done = true }) // mutate with a function
```

The implementation picks the target of the change: for an options store it's the reactive state `pinia.state[id]`, for a setup store it's the store itself (a write through `proxyRefs` lands in the `.value` of the right ref). One change, one path.

\$subscribe runs a callback on any state change. It's a watch on top of our reactivity:

```
store.$subscribe(() => console.log('store changed'))
```

A subtlety a test exposed: for a setup store you can't watch both the state and the getters at once. Otherwise a single change to `items` would fire twice — once from the ref itself and once from the computed that depends on it. So we filter out the computed (they carry a `.effect`) and watch only the state fields. A small detail, but it shows how the layers connect: the store subscription leans on how computed from layer 1 is built.

\$reset returns the state to its initial values. It works for options stores, where the `state()` function that produces fresh values is known. A setup store has no initial snapshot, so `$reset` honestly warns that it's unavailable.

The final app

playground/12-capstone.html is a “Notes” app where everything runs at once:

- **Router** — three routes: the list /, editing /note/:id with a param, and an “About” page. Navigation through RouterLink, rendering through RouterView.
- **Store** — useNotes in setup style holds the notes; storeToRefs hands reactive refs to the component; \$subscribe logs changes to the console.
- **Forms** — adding and editing through v-model, submitting on @keyup.enter.
- **Teleport** — the delete confirmation is moved into a modal above the page.
- **Components** — the list and the detail talk only through the store and the router, with no props passed between them.

Not a single third-party line: reactivity, rendering, template compilation, the router, and the store — all ours, assembled over twelve layers. If this app works, the whole framework works.

What we covered

Twelve layers, each built on the ones before it:

1. **Reactivity** — track/trigger, ref, reactive, computed, watch.
2. **Virtual DOM** — h, the renderer, keyed diff via the longest increasing subsequence.
3. **Components** — setup, the reactive render effect, the scheduler, props/emit, slots, provide/inject, the lifecycle, createApp.
4. **Compiler** — from a template to a render function through with(ctx).
5. **Router** — reactive route, history, params, guards.
6. **Store** — defineStore, getters on computed, actions, plugins.
7. **SSR** — renderToString and hydration by adopting the DOM.
8. **Forms** — v-model, class/style binding, event modifiers.
9. **Reactivity extensions** — watchEffect, readonly, shallow*, markRaw.
10. **Directives and dynamics** — custom directives, <component :is>, v-model on components.
11. **Built-in components** — Teleport, KeepAlive, async components.
12. **Store extensions** — \$patch, \$subscribe, \$reset — and this app.

All of them grew from the single idea of chapter one: “the data changed, so whatever depends on it updated itself.” Reactivity turned out to be not one feature among many but the foundation everything else stands on — rendering, components, the router, the store, even SSR. That’s the book’s main takeaway: big systems rarely rest on many complex mechanisms. More often it’s one right idea, carried carefully into every corner.

What's next

Real Vue differs from ours not in ideas but in finish: patch flags to speed up the diff, static hoisting in the compiler, Suspense and streaming SSR, devtools, strict typing, thousands of tested edge cases. All of it is engineering polish over exactly the mechanisms you've now written yourself. Open Vue's source: you'll find familiar `track`, `trigger`, `patchKeyedChildren`, `setupComponent` — just larger and more careful. You'll read them now not as magic but as an extended version of your own code.

Thanks for going the whole way. You didn't learn Vue — you wrote it.

Check yourself

```
npm test          # all 96 tests across twelve layers
npm run serve     # http://localhost:5173/playground/12-capstone.html
```

The “Notes” app is the final check: add a note, open it, edit it, delete it through the modal, click through the routes. It all runs on a framework that twelve chapters ago was an empty folder.